HAND WAVE QUIZ: PLAY AND ANSWERS WITH GESTURES

MINI PROJECT

Submitted to

Jawaharlal Nehru Technological University, Hyderabad

In partial fulfillment of the requirements for the

Award of the degree of

BACHELOR OF TECHNOLOGY

In

CSE (DATA SCIENCE)

By

J.RAVI TEJA (225U1A6720)

T.NISAHNTH (225U1A6756)

K.NITHIN REDDY (225U1A6723)

LAXMAN (225U1A6750)

Under the Guidance of

Dr.REVATHI DURGAM

ASSOCIATE PROFESSOR

AVN INSTITUTE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF CSE (DATA SCIENCE)

(Affiliated to JNTUH, Approved by A.I.C.T.E, Accredited by NAAC, New Delhi) Koheda Road, Ramdaspally, Ibrahimpatnam Ranga Reddy District – 501510, Telangana, India **2024-25**

AVN INSTITUTE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF CSE (DATA SCIENCE)

CERTIFICATE

This is to certify that the Mini Project Report on ***" HAND WAVE QUIZ:PLAY AND ANSWERS WITH GESTURES "*** submitted by **J.RAVI TEJA, T.NISHANTH,K.NITHIN REDDY,S.LAXMAN** bearing Hallticket Nos. **225U1A6720 ,225U1A6756, 225U1A6723, 225U1A6750,** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **CSE (DATA SCIENCE)** from Jawaharlal Nehru Technological University, Kukatpally, Hyderabad for the academic year 2024-25 is a record of Bonafide work carried out by them under our guidance and supervision.

Internal Guide	**Head of the Department**
Dr.Revathi Durgam	**Dr.Revathi Durgam**
Associate Professor	**Associate Professor**

External Examiner

AVN INSTITUTE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF CSE (DATA SCIENCE)

DECLARATION

We **J.Ravi Teja,T.Nishanth,k.Nithin Redddy,S.Laxman** bearing Hall ticket Nos. **225U1A6720, 225U1A6756, 225U1A6723, 225U1A6750,** hereby declare that the Mini Project title **HAND WAVE QUIZ:PLAY AND ANSWERS WITH GESTURES** done by us under the guidance of **Dr.Revathi Durgam , Associate Professor** which is submitted in the partial fulfillment of the requirement for the award of the B. Tech degree in **CSE (DATA SCIENCE)** at AVN Institute of Engineering & Technology for Jawaharlal NehruTechnological University, Hyderabad is our original work.

J.RAVI TEJA	**225U1A6720**
T.NISHANTH	**225U1A6756**
K.NITHIN REDDY	**225U1A6723**
S.LAXAM	**225U1A6750**

AVN INSTITUTE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF CSE (DATA SCIENCE)

ACKNOWLEDGEMENT

The successful completion of any task would we incomplete without mention of the people who made it possible through their guidance and encouragement crowns all the efforts with success.

We take this opportunity to acknowledge with thanks and deep sense of gratitude to **Dr.Revathi Durgam,Associate Professor, CSE (DATA SCIENCE)** for her constant encouragement and valuable guidance during the project work.

A special vote of thanks to **Dr.Revathi Durgam**, **Project Coordinator** , **Dr.Revathi Durgam, Head of the Department** and **Dr. P Nageshwara Reddy, Principal,** who have been a source of continuous motivation and support. They had taken time and effort to guide and correct us all through the span of this work. We owe very much to the **Department Faculty** and the **Management** who made our term at AVN Institute of Engineering and Technology a stepping stone for our career. We treasure every moment we had spent in the college.

Last but not the least, our heartiest gratitude to our parents and friends for their continuous encouragements and blessings. Without their support this work would have not been possible.

ABSTRACT

The HandWave Quiz is an innovative, gesture-controlled quiz system designed to enhance user interaction without relying on traditional input devices such as keyboards or touchscreens. This project utilizes computer vision technologies—specifically OpenCV and MediaPipe—to detect both the user's face and hand gestures in real time. Users interact with the quiz by raising one or two fingers, which are mapped to specific answer choices, allowing for seamless, touch-free participation.

The interface displays the user's face alongside dynamically presented quiz questions, creating an immersive and interactive experience. Immediate feedback and real-time score updates keep users engaged throughout the quiz. The system's accessibility makes it highly suitable for educational environments, training modules in corporate sectors, and public kiosks. It requires only a standard webcam, making the setup cost-effective and widely deployable.

With a strong emphasis on hygiene, user engagement, and accessibility, the HandWave Quiz exemplifies a modern approach to human-computer interaction. Future development plans include the integration of voice recognition and machine learning algorithms for personalized question generation, further enhancing user experience. This project demonstrates the potential of combining artificial intelligence, computer vision, and intuitive design to revolutionize interactive learning and gaming platforms.

TABLE OF CONTENTS

Chapter 1: INTRODUCTION ……..……1

1.1 Introduction …….…….....1

1.2 Motivation …….……..1

1.3 Objective …….…….……2

1.4 Scope …….……...2

Chapter 2: LITERATURE SURVEY …….……....3

2.1 Literature Review ……...….…3

2.2 Overview of Survey Methods …….….3

2.3 Existing Technologies …….....4

2.4 Identified Gaps and Key Challenges ……..5

Chapter 3: PROPOSED SYSTEM …….....7

3.1 Existing System and its Limitations …….....7

3.2 Functional Requirements ……....8

3.3 Non – Functional Requirements …….....8

3.4 Proposed System …….....9

3.4.1Multilingual Support and Language Detection **……..9**

3.4.2 Emotion and Sentiment Analysis **……..9**

3.4.3 Visual Feedback and Emotion-to-Image Mapping……..9

3.4.4 Bulk Analysis and Real-Time Integration……..9

Chapter 4: SYSTEM DESIGN …….....10

4.1 Importance of Design ……...10

4.2 System Architecture ……11

4.3 UML Diagrams …….....11

4.3.1 UML Use case Diagram …….....12

4.3.2 UML Class Diagram ……...12
4.3.3 UML Sequence Diagram …….....14
4.3.4 UML Activity Diagram ……..16
Chapter 5: IMPLEMENTATION ……...18
5.1 Module Description ……..18
5.2 Sample Code ……..……..19
Chapter 6: TESTING ……24
6.1 Importance of Testing ……...24
6.2 Testing Details ...…… …….24
6.3 Test Cases …….25
Chapter 7: OUTPUT SCREENSHOTS ……..27
Chapter 8: CONCLUSION AND FUTURE SCOPE …….31
Chapter 9: REFERENCES ……...33

LIST OF FIGURES

SL.NO	Fig. No.	Title	Page no.
1	3.1	Existing System	8
2	3.1.2	Proposed System	9
3	4.1.1	System Architecture	11
4	4.1.2	UML UseCase	12
5	4.1.3	UML Class Diagram	14
6	4.1.4	UML Sequence Diagram	16
7	4.1.5	Activity Diagram	17
8	7.1	Login	27
9	7.2	Invalid Login	28
10	7.3	Instruction	28
11	7.4	Face Not Detected	29
12	7.5	Selecting Answers	29
13	7.6	Exit and Final Score	30

LIST OF TABLES

Sl.no.	Table no.	Title	Page no.
1	6.1	Test Details	23
2	6.2	Test Cases	26

CHAPTER 1

INTRODUCTION

1.1 Introduction

The *HandWave Quiz* is a gesture-controlled, touch-free quiz platform that transforms the traditional question-and-answer format into an engaging, interactive experience using computer vision. This system eliminates the need for physical input devices such as keyboards or touchscreens by allowing users to interact through simple hand gestures detected via a webcam.

Built using OpenCV and MediaPipe, the application can detect both the user's face and hand gestures in real-time. Users are prompted with quiz questions and can respond by showing specific finger gestures—such as raising one or two fingers—to select their answers. The interface includes visual feedback and live score updates, making the entire experience dynamic and user-friendly.

This approach to quiz participation not only enhances engagement but also supports accessibility and hygiene, making it ideal for environments like classrooms, training sessions, and public kiosks. It serves as an example of how modern human-computer interaction can be intuitive, inclusive, and impactful.

1.2 Motivation

The idea behind the *HandWave Quiz* was motivated by the increasing need for touchless technologies, especially in the wake of the COVID-19 pandemic, where minimizing contact is crucial. Traditional quiz systems, often reliant on keyboards or touchscreens, present hygiene risks and may not be accessible to users with physical limitations Additionally, there is a rising demand for interactive and smart learning tools that make education more engaging and enjoyable. By using hand gestures, users can interact naturally with the system without requiring specialized skills or equipment. The goal was to build a system that bridges the gap between usability, innovation, and inclusion—paving the way for smarter learning and assessment tools.

Additionally, there is a rising demand for interactive and smart learning tools that make education more engaging and enjoyable. By using hand gestures, users can interact naturally with the system without requiring specialized skills or equipment. The goal was to build a system that bridges the gap between usability, innovation, and inclusion—paving the way for smarter learning and assessment tools.

1.3 Objective

The main objectives of the *HandWave Quiz* project are:

- To develop a real-time quiz system controlled by hand gestures.
- To detect hand gestures and face presence using OpenCV and MediaPipe.
- To allow touch-free answering of multiple-choice questions.
- To provide immediate visual feedback and real-time score updates.

- To create an accessible and cost-effective solution for interactive learning.
- To demonstrate the potential of computer vision in educational applications.
- To lay a foundation for future enhancements such as AI-based adaptive quizzes and voice integration.

1.4 Scope

The scope of this project encompasses the design, development, and implementation of a basic gesture-controlled quiz system. It supports:

- Gesture recognition for answer selection (e.g., 1 finger = Option A, 2 fingers = Option B).
- Face detection to ensure user presence before answering.
- A graphical user interface built using Tkinter for quiz display and interaction.
- Real-time processing using a standard webcam, without the need for external sensors.
- Input validation, feedback alerts (e.g., “No face detected”), and final scoring at the end.

The system is currently limited to basic gestures and two answer options but can be extended further. Future expansions may include:

- Support for more gestures and more answer choices.
- Voice recognition for voice-controlled quizzes.
- Integration of AI for personalized question recommendations.
- Use in mobile platforms and smart classrooms.

The project showcases how intuitive, low-cost, and scalable technology can be leveraged to improve learning and engagement in various environments.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Review

Human-computer interaction (HCI) has evolved rapidly with the emergence of computer vision and artificial intelligence, enabling more natural and intuitive interfaces. Among these, gesture recognition systems have gained significant attention due to their ability to facilitate touchless interaction. In educational and public applications,

such systems offer hygienic, accessible, and user-friendly alternatives to conventional input methods like keyboards or touchscreens.

Several studies have explored gesture recognition using technologies such as OpenCV, MediaPipe, and deep learning-based models. OpenCV is widely used for image processing tasks, while MediaPipe provides efficient solutions for real-time hand and face landmark detection. Projects combining these tools have been used in areas like virtual mouse control, sign language interpretation, and gaming interfaces.

In the context of quiz applications, most existing systems are designed for mouse, keyboard, or touchscreen inputs. These traditional models often lack accessibility features and are not optimized for contactless interaction. While voice-assisted quizzes exist, they are limited by environmental noise and speech recognition accuracy.

Recent research suggests that vision-based gesture recognition provides a more stable alternative in controlled environments. Systems that detect hand gestures—like raising fingers or swiping—can be effectively mapped to specific actions, making them suitable for use in quiz applications. However, there is limited implementation of gesture-based quizzes in real-world educational tools.

This gap highlights the need for an intuitive, gesture-controlled quiz platform that is easy to use, requires minimal hardware, and supports interactive learning. The *HandWave Quiz* aims to address this gap by integrating real-time hand and face detection with a quiz engine, providing a novel learning experience through natural user interaction.

2.2 Overview of Survey Methods

To ensure that the development of the *HandWave Quiz* addressed real-world needs and challenges, a survey-based approach was undertaken to gather insights from target users and analyze

the effectiveness of gesture-based interaction in quiz environments.

User-Centric Approach

A qualitative survey was conducted among students, teachers, and general users to assess their familiarity with gesture recognition technologies and preferences for interaction in learning environments. The survey focused on:

- Current methods used for taking quizzes (keyboard, mouse, mobile apps).
- Awareness of gesture-controlled systems.
- Comfort and interest in using hand gestures for answering questions.
- Perceived advantages and limitations of contactless interaction.
- Preferred features in a quiz application (real-time feedback, scoring, etc.).

Feedback revealed that users found gesture-based input both engaging and hygienic, especially in group settings like classrooms or training centers. Many users also expressed interest in a system that works without physical contact or typing, especially for public kiosks or during health-sensitive times (e.g., during a pandemic).

Technology Analysis

In parallel, technical surveys were conducted to explore existing tools and frameworks used in gesture detection. Libraries like OpenCV and MediaPipe were evaluated for:

- Real-time performance with standard webcams.
- Accuracy in detecting hand landmarks and face presence.
- Ease of integration with GUI libraries like Tkinter.
- Suitability for offline, cost-effective.

2.3 Existing Technologies

The development of gesture-controlled systems has been made possible through advancements in computer vision and machine learning. Several technologies currently exist that support the detection of facial and hand landmarks, enabling intuitive and touch-free human-computer interaction. The *HandWave Quiz* builds upon such technologies to deliver a responsive and interactive quiz experience.

OpenCV (Open Source Computer Vision Library)

OpenCV is an open-source computer vision library widely used for image processing, object detection, and real-time computer vision applications. It provides efficient tools for working with live video streams, such as capturing frames from a webcam and applying filters or detection algorithms. In this project, OpenCV is used to process camera input, extract image frames, and interface with gesture detection modules.

MediaPipe

Developed by Google, MediaPipe is a cross-platform framework for building multimodal machine learning pipelines. It offers robust pre-trained models for hand tracking, face detection, and pose estimation. For the *HandWave Quiz*, MediaPipe's **Hands** and **Face Detection** modules are used to detect hand landmarks and ensure user presence before allowing input. These modules operate in real-time and provide high accuracy even under normal lighting conditions with a standard webcam.

Tkinter

Tkinter is the standard GUI library for Python and is used to design the interface of the quiz application. It allows for the creation of windows, buttons, labels, and dynamic content such as quiz questions and score displays. Its lightweight nature and integration with Python make it ideal for desktop-based educational tools.

Python Programming Language

Python serves as the primary development language due to its simplicity, vast library support, and suitability for rapid prototyping. It acts as the backbone of the application logic, coordinating video processing, gesture interpretation, question flow, and GUI rendering.

These technologies, when integrated effectively, create a seamless experience for users to interact with quiz content using only gestures and facial presence—eliminating the need for traditional input devices.

2.4 Identified Gaps and Key Challenges

Despite significant progress in gesture recognition and interactive quiz systems, several gaps and challenges remain, which the *HandWave Quiz* aims to address.

Limited Adoption of Gesture-Based Quizzes

Most existing quiz applications rely on traditional input methods such as keyboards, mice, or touchscreens. While some experimental systems utilize voice commands or gestures, widespread adoption is limited due to technical complexities and lack of user-friendly implementations. This gap highlights the need for accessible, reliable gesture-controlled quiz platforms.

Real-Time Accuracy and Robustness

Accurate real-time detection of hand gestures and face presence is critical for usability. Variations in lighting, background clutter, and hand positioning can affect detection accuracy. Many existing solutions require specialized hardware or controlled environments, limiting their practicality. Achieving consistent performance with a standard webcam remains a challenge.

User Accessibility and Inclusivity

Current systems often overlook accessibility considerations, such as support for users with limited mobility or varying hand sizes. Furthermore, gesture sets must be intuitive and easy to perform to ensure broad usability. Designing a system that caters to diverse users without overwhelming them is essential.

Integration and Feedback Mechanisms

Effective quiz systems must provide immediate, clear feedback to maintain engagement and learning efficacy. Existing gesture-based systems sometimes lack seamless integration of detection, interface, and feedback, leading to a disjointed user experience. Real-time score updates, error handling (e.g., “face not detected”), and user prompts are necessary components that require careful design.

Scalability and Extensibility

Many prototype systems support only limited gestures or question types. Expanding these platforms to handle multiple-choice questions with varying options, adaptive difficulty, and multimedia content is still an open area. Ensuring scalability while maintaining responsiveness is challenging.

CHAPTER 3

PROPOSED SYSTEM

The **HandWave Quiz: Play and Answer with Gestures** is an interactive quiz system that leverages real-time computer vision to enhance user engagement and security through gesture recognition and face detection. It eliminates the need for traditional input devices such as keyboards or mice by allowing users to answer multiple-choice questions using simple hand gestures. The system uses **MediaPipe** for hand tracking and **OpenCV** for video processing, enabling users to point and select answers with their index finger, providing a contactless and intuitive interaction.

Face detection is integrated to ensure the presence of the authenticated user throughout the quiz, reducing the chances of impersonation and cheating. The application starts with a login interface where users enter their name and roll number, after which a full-screen quiz interface is launched. Each question is displayed along with four options, and the system monitors hand movement to detect which option the user is pointing to. A selection is confirmed when the user holds the gesture over an option for more than two seconds.

The proposed system runs in real-time with minimal latency and provides a smooth user experience. Upon quiz completion, the application displays a score summary with the user's details in a GUI-based window. Built using **Python**, **Tkinter**, and **MediaPipe**, the application is lightweight, easy to deploy, and designed to work on any system with a webcam.

Future upgrades may include voice-based feedback, gesture customization, integration of difficulty levels, and storing results in a database for analytics. This system is especially suitable for smart classrooms, e-learning platforms, and interactive training environments, promoting inclusive, engaging, and modern quiz-taking experiences.

3.1 Existing System and Its Limitations

The traditional quiz systems primarily rely on keyboard or mouse inputs and often lack accessibility features. Existing platforms do not support gesture-based interactions or real-time monitoring. Users with physical disabilities face barriers in using such interfaces. Moreover, these systems do not ensure user authenticity or prevent impersonation during assessments. Face presence and gesture-based answering are absent, reducing engagement and security.

. **3.2 Functional Requirements**

- **Login & Authentication**: Secure login system using username and password.
- **Hand Gesture Detection**: Real-time tracking of finger counts to select quiz options.
- **Face Detection**: Ensures the user is present during the quiz.
- **Quiz Flow Management**: Display questions, navigate, record responses, and compute score.
- **Feedback System**: Display final score after quiz completion.
- **Error Handling**: Prompt for missing input or absence of face/hand in frame.

3.3 Non-Functional Requirements

- **Usability**: The interface must be simple, intuitive, and user-friendly.
- **Performance**: Real-time processing of webcam feed with minimal delay.
- **Reliability**: System should consistently detect gestures and faces under varying conditions.
- **Security**: User data and quiz responses should be securely handled.
- **Portability**: Should run on standard Windows/Linux PCs with a webcam.

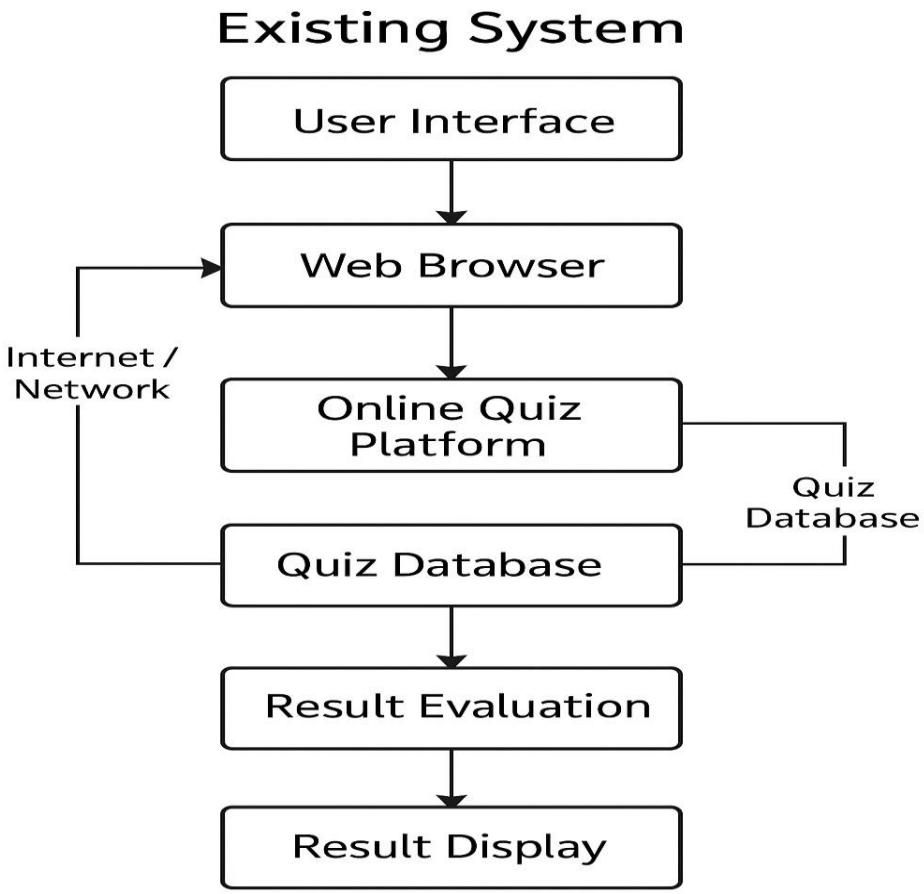

fig.3.1 Existing System

3.4Proposed System

The proposed *HandWave Quiz* system integrates computer vision with a GUI-based interface to offer a touchless quiz experience. It uses OpenCV and MediaPipe for hand gesture recognition and face detection, making the process more interactive and inclusive. Tkinter is used for GUI design to support user login and quiz flow.

3.4.1Multilingual Support and Language Detection

Though not currently implemented, future enhancements will include language detection and support for multiple languages to widen accessibility.

3.4.2 Emotion and Sentiment Analysis

The system can be extended to monitor facial expressions to assess confidence or stress levels during the quiz, providing psychological feedback or adaptive questioning.

3.4.3 Visual Feedback and Emotion-to-Image Mapping

Adding real-time emotional response visualization (smiley, neutral, sad) based on detected expressions could help in making the user experience more engaging and human-like.

3.4.4 Bulk Analysis and Real-Time Integration

Future development may include dashboards for instructors to monitor multiple students' progress, face detection status, and quiz results in real-time during online assessments.

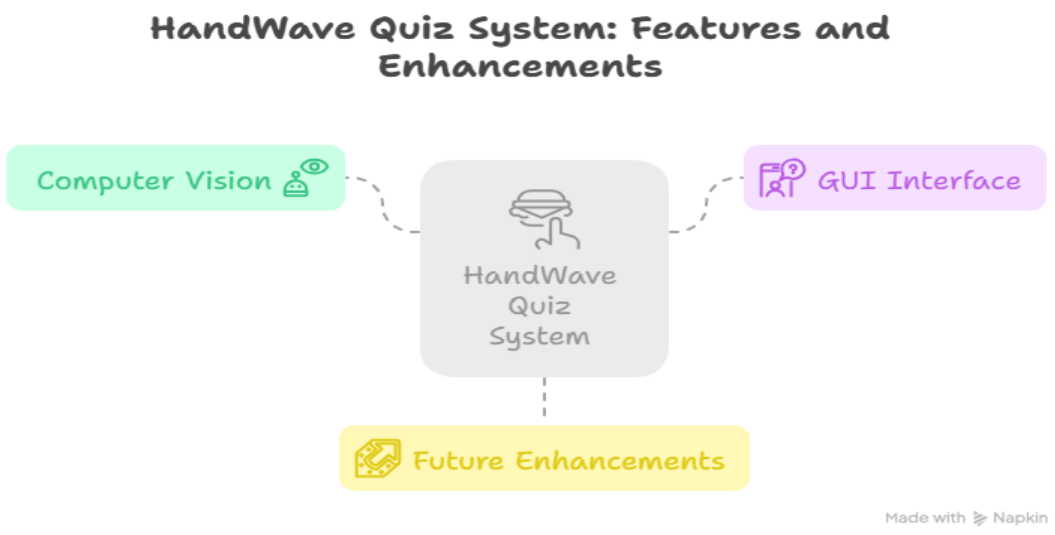

fig.3.1.2 Proposed System

CHAPTER 4

SYSTEM DESIGN

4.1 Importance of Design

Design is a fundamental phase in the software development life cycle that bridges the gap between requirements and implementation. It outlines how the system should behave, how different modules will interact, and how data will flow throughout the system. A strong design helps to minimize complexity, promotes code reusability, and ensures that the system meets both functional and non-functional requirements.

In the case of the *HandWave Quiz* system, design was crucial in determining how various technologies like OpenCV, MediaPipe, and Tkinter

4.2 System Architecture

The architecture of the *HandWave Quiz* system is designed with modularity, extensibility, and real-time responsiveness in mind. The system follows a layered architecture that includes the following components:

- **User Layer**: The end-user interacts with the system using simple hand gestures in front of a webcam.
- **Input Capture Module**: A webcam continuously captures real-time video frames.
- **Computer Vision Engine**: This is the core processing unit that utilizes **OpenCV** and **MediaPipe** for:
    - **Hand Gesture Recognition** – Identifying finger counts to select answers.
    - **Face Detection** – Checking whether the user is properly positioned.
- **Application Logic (Quiz Engine)**:
    - Presents multiple-choice questions.
    - Verifies answers based on gesture input.
    - Tracks score and quiz progress.
- **GUI Layer**: Built using **Tkinter**, this layer includes:
    - Login and registration interfaces.
    - Quiz screens with real-time feedback.
    - Score summary and logout.
- **Optional Future Extensions**:
    - **Emotion Detection**: To gauge stress/confidence.
    - **Multilingual Interface**: To support users of various languages.
    - **Instructor Dashboard**: For real-time student monitoring.

would work together. It ensured that the user interface is intuitive, the gesture recognition is accurate, and the face detection system is reliable — creating a seamless and smart quiz-taking experience.

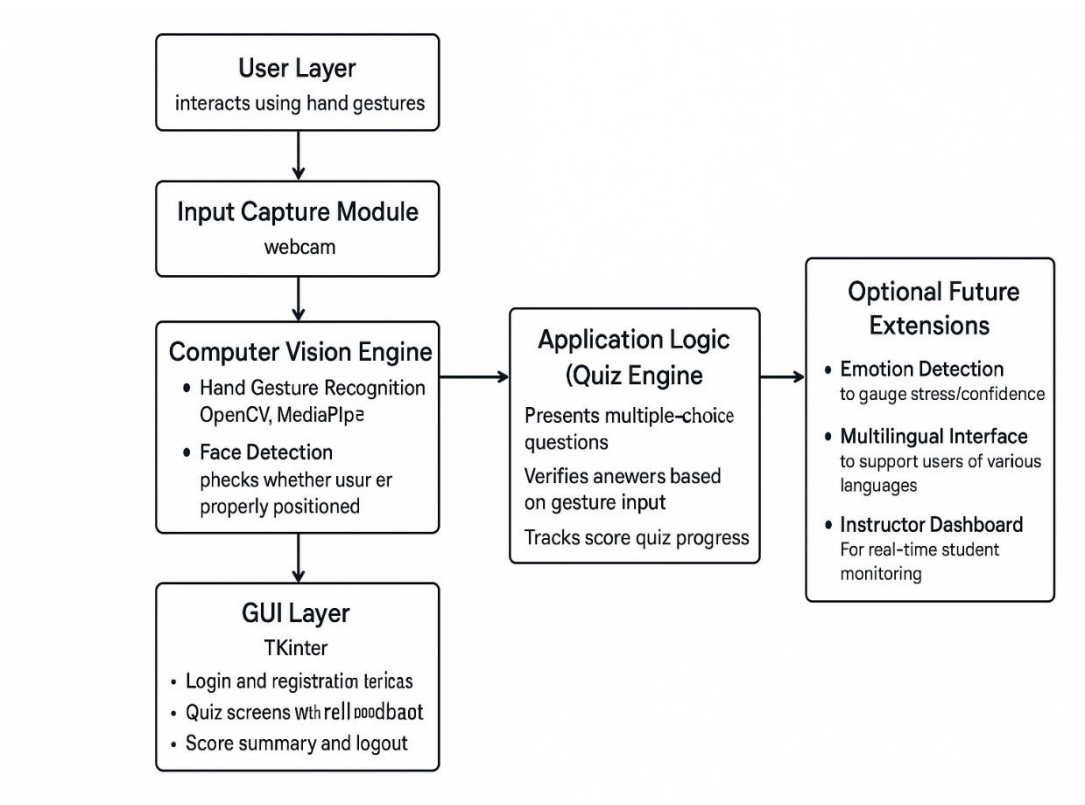

fig.4.1.System Architecture

4.3 UML Diagrams

Unified Modeling Language (UML) diagrams play a vital role in visualizing, designing, and documenting the architecture of the system. In the case of the *Smart Retail System* (or any intelligent software solution), these diagrams help represent both the structural and behavioral aspects in a clear and organized manner.

- **Use Case Diagram**: This provides a high-level overview of the system by identifying key **actors** such as the *Store Owner* and their interactions with essential functionalities. These include adding new products, generating bills, receiving low-inventory alerts, and analyzing sales trends. It helps define the system's scope and user requirements.
- **Class Diagram**: This diagram illustrates the static structure of the system by detailing classes, attributes, methods, and their relationships. For instance, a VoiceAssistant class might include methods for recognizing speech and mapping it to product categories, while an InventoryManagement class would manage stock levels, trigger low-stock alerts, and suggest restocking actions.
- **Sequence Diagram**: It represents the dynamic behavior of the system by modeling how objects interact over time to complete a process. An example scenario could be the addition of a product using voice input, where the VoiceAssistant captures the command, updates the InventoryManager, and passes relevant data to the AnalyticsModule for future forecasting.

Together, these UML diagrams offer a holistic view of the system's functionality and interactions. They serve as essential tools for developers, enabling better understanding, implementation, testing, and scalability of the software solution.

4.3.1.UML Usecase Diagram

The Use Case Diagram represents the interaction between the user and the system. It highlights the key functionalities provided to the user such as:

- Login/Register
- Start Quiz
- Answer via Hand Gestures
- Real-Time Face Detection
- View Score and Feedback

This helps in identifying the main features and user expectations from the system.

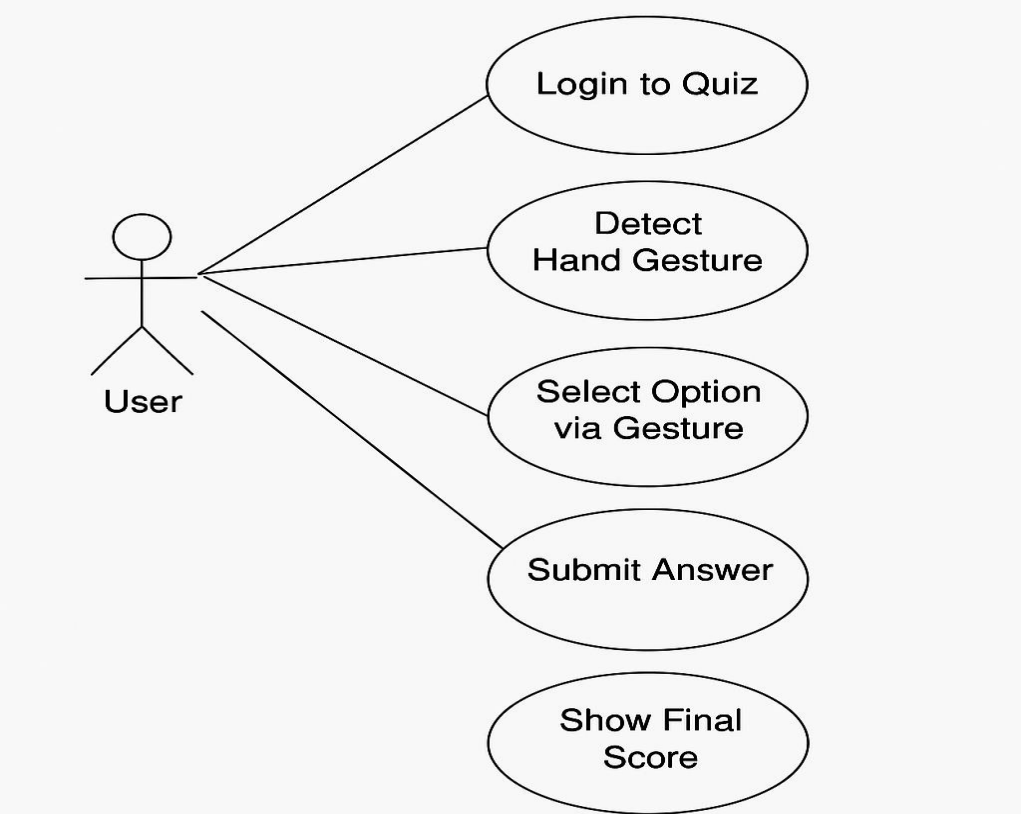

fig.4.1.2.UseCase Diagram

4.3.2 UML Class Diagram

The UML Class Diagram illustrates the static structure of the HandWave Quiz system by representing its core classes, their attributes, operations (methods), and relationships among them. This diagram helps in understanding the object-oriented design and encapsulation of logic within different modules of the system.

Class Descriptions:

- **User**:
  Responsible for handling login functionalities. It includes attributes like username and password, and operations for logging in and out.
- **Question**:
  Represents a quiz question. It stores the question text, available options, and the correct answer. It provides methods to retrieve the question and validate if a given answer is correct.
- **QuizManager**:
  Acts as the controller of the quiz flow. It manages the list of questions, the current question index, and the user's score. It coordinates with other classes to move through the quiz and update scores.
- **GestureRecognizer**:
  This class uses computer vision libraries like MediaPipe or OpenCV to detect hand gestures. It extracts landmark points and maps the fingertip location to a selected answer option.
- **AnswerEvaluator**:
  Validates the user's selected answer against the correct option. It returns a boolean result used to update the score.
- **UIController**:
  Manages the graphical interface using libraries such as Tkinter. It is responsible for displaying questions, highlighting selected answers based on gestures, and presenting the final result.

Relationships:

- The User interacts with the QuizManager to initiate the quiz.
- The QuizManager communicates with Question, GestureRecognizer, AnswerEvaluator, and UIController.
- GestureRecognizer interprets user gestures and sends the mapped option back to the QuizManager.
- AnswerEvaluator checks the correctness of the selected option.
- UIController renders the interface and responds to user interactions.

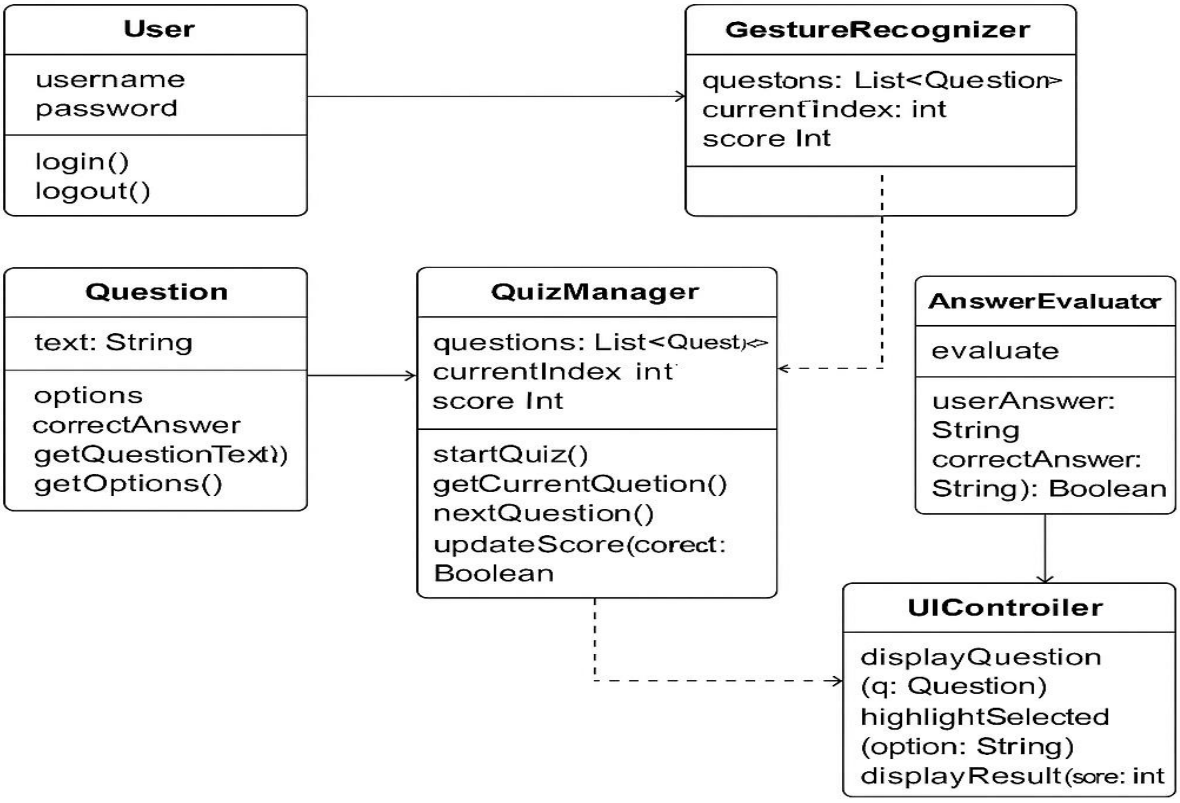

fig.4.1.3.Class Diagram

4.3.3 UML Sequence Diagram

The UML Sequence Diagram for the HandWave Quiz system demonstrates the dynamic behavior of the application by capturing the interaction between various components over time. It highlights how objects exchange messages during the execution of a quiz session, particularly focusing on the gesture-based answer selection mechanism.

Flow Description:

1. User initiates the login process through the GUI.
2. Upon successful authentication, the GUI triggers the QuizManager to start the quiz.
3. For each question:
    - The GUI requests a video frame from the Camera.
    - The Camera sends this frame to the GestureRecognizer, which processes it using hand landmark detection.
    - The GestureRecognizer identifies the selected answer based on fingertip location and sends it back to the GUI.
4. The selected answer is forwarded by the GUI to the QuizManager, which in turn calls the AnswerEvaluator to check correctness.

5. The AnswerEvaluator returns a boolean value indicating whether the answer is correct.
6. The QuizManager updates the score and sends the result to the GUI, which updates the display accordingly.
7. This loop continues until all questions are answered.
8. Finally, the GUI displays the final score and ends the quiz session.

Objects Involved:

- User
- GUI (Tkinter-based)
- Camera (OpenCV)
- GestureRecognizer (MediaPipe/OpenCV)
- QuizManager
- AnswerEvaluator

Purpose:

This diagram effectively captures the sequence of interactions required to operate the contactless quiz system. It provides a clear understanding of the order of operations, message flow, and roles of different modules, making it easier to implement, test, and debug the application.

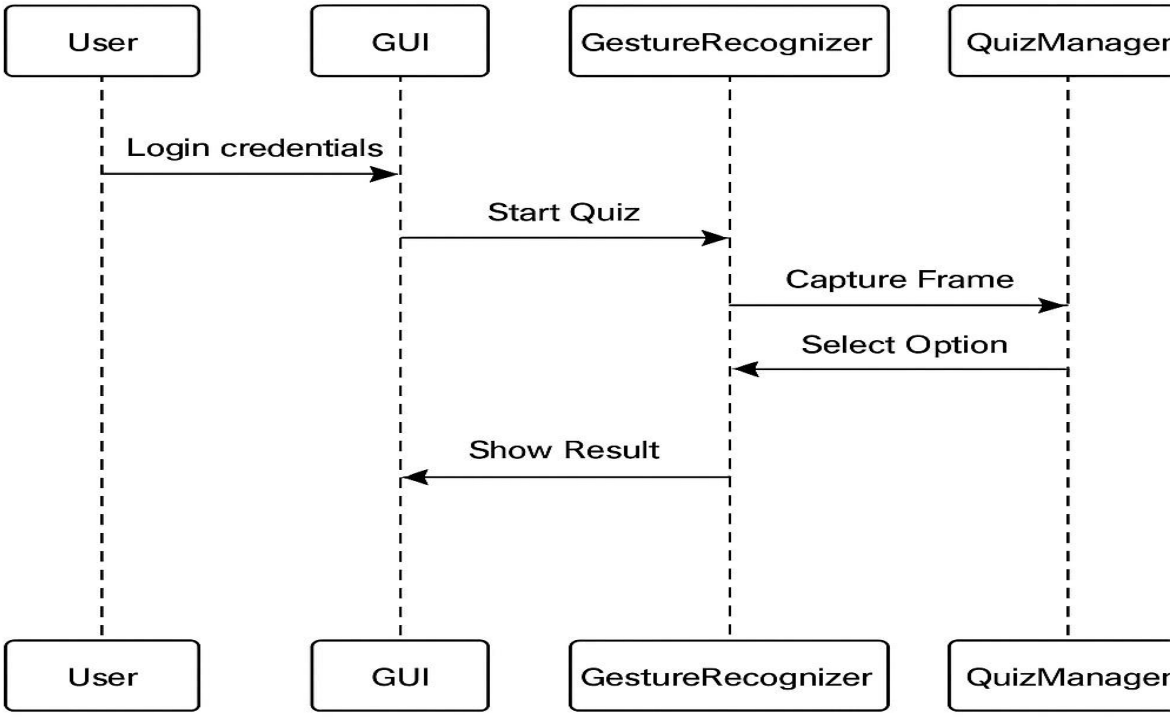

fig.4.1.4.Sequence Diagram

4.3 UML Activity Diagram Description

The Activity Diagram illustrates the step-by-step workflow of the HandWave Quiz system. It starts with user authentication and proceeds through the following key actions:

- **User Login**: The process begins when the user logs into the system.
- **Start Quiz**: Upon successful login, the quiz is initialized.
- **Show Question**: A quiz question along with multiple-choice answers is displayed.
- **Face Detection Check**: Before accepting a response, the system verifies whether the user's face is correctly positioned. If not, an alert is shown ("Face is not detected").
- **Continue Quiz**: If the face is detected, the quiz proceeds.
- **Place Answer by Hand Gesture**: The user selects an answer using hand gestures tracked by the system.
- **Next Question**: The next question is loaded in a loop until the quiz is completed.
- **Show Final Score**: After all questions are answered, the final score is displayed.
- **Exit**: The process

This diagram helps visualize the dynamic behavior and flow control of the quiz system, particularly its real-time interaction with gesture-based answering and face detection. s ends.

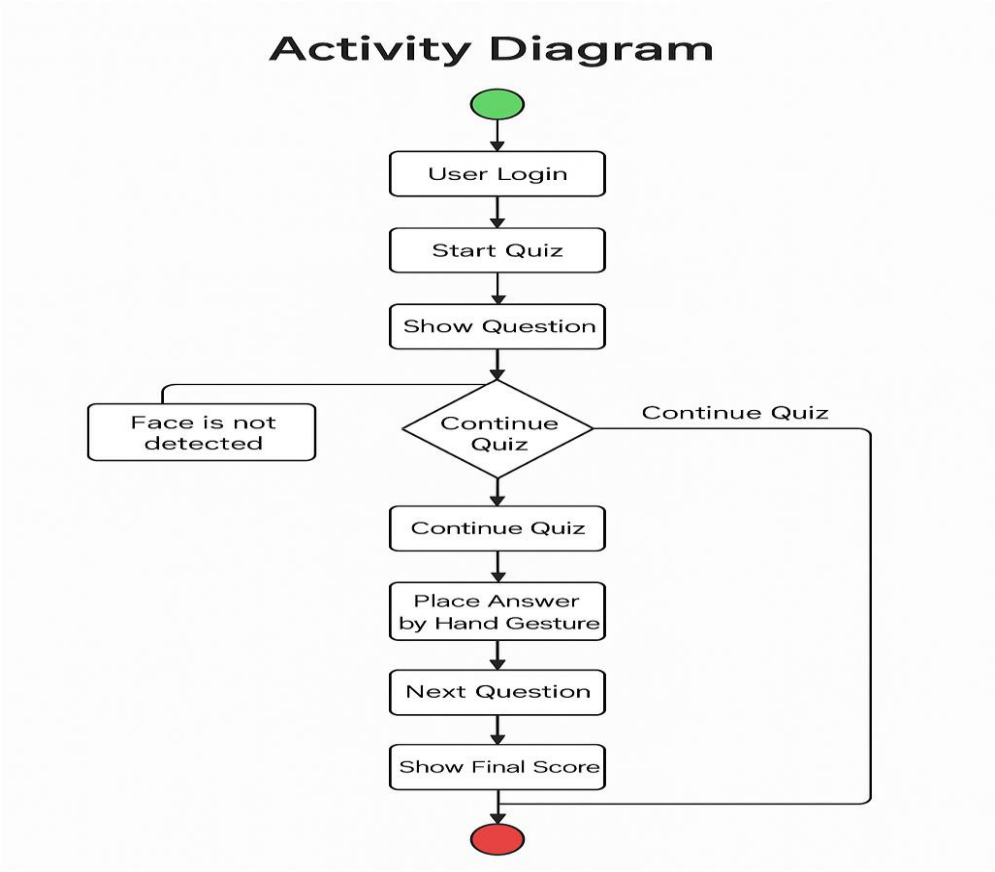

Fig.4.1.5.Activity Diagram

CHAPTER 5

IMPLEMENTATION

5.1 Module Description

The implementation of the *HandWave Quiz: Play and Answer with Gestures* system is divided into several key modules to ensure modularity and ease of understanding. Each module is designed to handle a specific functionality within the application.

1. User Authentication Module:
This module handles user login by validating the username and password input fields. It ensures only registered users can access the quiz.

2. Quiz Interface Module:
Responsible for displaying the quiz questions and multiple-choice answers. It manages the navigation between questions and updates the user interface accordingly.

3. Hand Gesture Recognition Module:
Utilizes MediaPipe and OpenCV to capture live webcam feed, detect the user's hand, and recognize specific finger gestures (e.g., one finger for option A, two fingers for option B). This module processes real-time video frames to identify the user's selected answer.

4. Face Detection Module:
Incorporates MediaPipe Face Detection to verify the user's presence during the quiz. It warns the user if the face is not detected to ensure integrity.

5. Score Calculation and Display Module:
Tracks the user's answers, calculates the total score based on correct responses, and displays the final result at the end of the quiz.

6. Error Handling and Validation Module:
Ensures input fields are not left empty, handles exceptions, and provides user feedback in case of invalid actions.

The integration of these modules enables a seamless and interactive quiz experience controlled through hand gestures.

5.2 Sample code

```
import cv2
import numpy as np
import mediapipe as mp
import time
import tkinter as tk
from tkinter import messagebox

# Initialize MediaPipe
mp_hands = mp.solutions.hands
mp_face = mp.solutions.face_detection
mp_drawing = mp.solutions.drawing_utils
hands = mp_hands.Hands(model_complexity=1, min_detection_confidence=0.8, min_tracking_confidence=0.8)
face_detection = mp_face.FaceDetection(min_detection_confidence=0.8)

# Global variables
user_name = ""
roll_number = ""
current_question = 0
score = 0
quiz_running = True
selected_option = -1
last_selection_time = 0

# Questions dataset
questions = [
    {"question": "Which keyword is used to create a class in Python?", "options": ["class", "define", "type", "struct"], "answer": 0},
    {"question": "What is the output of: bool('False')?", "options": ["False", "True", "None", "Error"], "answer": 1},
    {"question": "Which function is used to get the type of a variable?", "options": ["typeof()", "type()", "getType()", "varType()"], "answer": 1},
    {"question": "Which one is a Python immutable data type?", "options": ["list", "dict", "set", "tuple"], "answer": 3},
    {"question": "What does the 'break' keyword do in loops?", "options": ["Skips next line", "Stops the loop", "Repeats loop", "Breaks variable"], "answer": 1},
    {"question": "Which module in Python is used for regular expressions?", "options": ["regex", "re", "pyregex", "pattern"], "answer": 1},
    {"question": "Which method is used to add an item to a list?", "options": ["add()", "insert()", "append()", "push()"], "answer": 2},
    {"question": "What does the 'is' operator check in Python?", "options": ["Value equality", "Type equality", "Object identity", "None"], "answer": 2},
    {"question": "Which keyword is used to handle exceptions?", "options": ["exception", "try", "raise", "catch"], "answer": 1},
```

```
    {"question": "What will '5 // 2' return?", "options": ["2", "2.5", "2.0", "3"], "answer": 0}
]

# Login Page
def login_page():
    login_window = tk.Tk()
    login_window.title("User Login")
    login_window.geometry("600x400")
    login_window.configure(bg="#d1c4e9")
    login_window.resizable(False, False)

    tk.Label(login_window, text="Quiz Game Login", font=("Arial", 20, "bold"), bg="#d1c4e9").pack(pady=20)
    tk.Label(login_window, text="Enter Name:", font=("Arial", 14), bg="#d1c4e9").pack()
    name_entry = tk.Entry(login_window, font=("Arial", 14), width=30)
    name_entry.pack(pady=10)
    tk.Label(login_window, text="Enter Roll Number:", font=("Arial", 14), bg="#d1c4e9").pack()
    roll_entry = tk.Entry(login_window, font=("Arial", 14), width=30)
    roll_entry.pack(pady=10)

    def validate_login():
        global user_name, roll_number
        user_name = name_entry.get().strip()
        roll_number = roll_entry.get().strip()
        if any(char.isdigit() for char in user_name) or not roll_number.isdigit():
            messagebox.showerror("Invalid Input", "Please enter valid name and roll number.")
        else:
            login_window.destroy()

        tk.Button(login_window, text="Start Quiz", font=("Arial", 14, "bold"), bg="#4CAF50", fg="white",
command=validate_login).pack(pady=20)
    login_window.mainloop()

login_page()

# Quiz settings
width, height = 1920, 1080
option_height = 140
option_spacing = 160
smoothing_factor = 0.7
prev_x, prev_y = 0, 0
cap = cv2.VideoCapture(0)
```

```
cap.set(3, width)
cap.set(4, height)

cv2.namedWindow("Quiz Game", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Quiz Game", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

def draw_ui(frame, question, options, selected_option):
    font = cv2.FONT_HERSHEY_SIMPLEX
    text_size = 2.0
    thickness = 4
    text_color = (0, 0, 0)

    (w, h), _ = cv2.getTextSize(question, font, text_size, thickness)
    x = (frame.shape[1] - w) // 2
    y = 100
    max_width = frame.shape[1] - 40

    words = question.split()
    line = ''
    y_offset = y
    for word in words:
        test_line = line + ' ' + word if line else word
        (test_w, _), _ = cv2.getTextSize(test_line, font, text_size, thickness)
        if test_w <= max_width:
            line = test_line
        else:
            cv2.putText(frame, line, (x, y_offset), font, text_size, text_color, thickness)
            y_offset += h + 10
            line = word
    cv2.putText(frame, line, (x, y_offset), font, text_size, text_color, thickness)

    option_boxes = []
    for i, option in enumerate(options):
        y_pos = 250 + (i * option_spacing)
        x1 = width // 6
        x2 = width * 5 // 6
        option_box = (x1, y_pos, x2, y_pos + option_height)
        option_boxes.append(option_box)
        color = (200, 200, 200) if selected_option != i else (0, 255, 0)
        cv2.rectangle(frame, (x1, y_pos), (x2, y_pos + option_height), color, -1)
        text_size_opt = cv2.getTextSize(option, font, 2.0, 4)[0]
```

```
        x_text = (width - text_size_opt[0]) // 2
        cv2.putText(frame, option, (x_text, y_pos + 90), font, 2.0, (0, 0, 0), 4)
    return option_boxes

def check_selection(x, y, option_boxes):
    for idx, (x1, y1, x2, y2) in enumerate(option_boxes):
        if x1 < x < x2 and y1 < y < y2:
            return idx
    return -1

# Main loop
while cap.isOpened() and quiz_running:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1)
    frame = cv2.resize(frame, (width, height))
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    face_results = face_detection.process(rgb_frame)

    if not face_results.detections:
        cv2.putText(frame, "Face not detected!", (600, 400), cv2.FONT_HERSHEY_SIMPLEX, 2, (0, 0, 255), 5)
        cv2.imshow("Quiz Game", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
        continue

    results = hands.process(rgb_frame)
    if current_question < len(questions):
        option_boxes = draw_ui(frame, questions[current_question]["question"], questions[current_question]["options"], selected_option)
    else:
        cap.release()
        cv2.destroyAllWindows()
        score_window = tk.Tk()
        score_window.title("Quiz Score")
        score_window.geometry("400x300")
        score_window.configure(bg="#e3f2fd")
        tk.Label(score_window, text=f"Quiz Over!\nName: {user_name}\nRoll No: {roll_number}\nScore: {score}/{len(questions)}", font=("Arial", 16, "bold"), bg="#e3f2fd").pack(pady=40)
        tk.Button(score_window, text="Exit", font=("Arial", 14, "bold"), bg="#FF5722", fg="white", command=score_window.destroy).pack(pady=20)
```

```
        score_window.mainloop()
        break

    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)
            index_tip = hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP]
            x, y = int(index_tip.x * width), int(index_tip.y * height)
            x = int(prev_x * smoothing_factor + x * (1 - smoothing_factor))
            y = int(prev_y * smoothing_factor + y * (1 - smoothing_factor))
            prev_x, prev_y = x, y
            cv2.circle(frame, (x, y), 20, (0, 0, 255), -1)
            current_selection = check_selection(x, y, option_boxes)
            if current_selection != -1:
                if current_selection != selected_option:
                    selected_option = current_selection
                    last_selection_time = time.time()
                elif time.time() - last_selection_time > 2:
                    if selected_option == questions[current_question]["answer"]:
                        score += 1
                    time.sleep(1)
                    current_question += 1
                    selected_option = -1

    cv2.imshow("Quiz Game", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

CHAPTER 6

TESTING

6.1 Importance of Testing

Testing is a fundamental phase in the development of any software system. For our project, *HandWave Quiz: Play and Answer with Gestures*, testing ensures that each component—from gesture recognition to score display—functions reliably, accurately, and efficiently.

Given that this is a gesture-based contactless system, testing not only verifies code correctness but also the real-time performance of the MediaPipe hand tracking module, OpenCV face detection, and Tkinter GUI rendering. We also tested environmental conditions such as lighting, camera angle, and background clutter, which directly influence the system's usability.

Benefits of testing in this project:

- Ensures accurate gesture-to-option mapping.
- Validates quiz logic and user flow.
- Identifies failures under multi-user or poor-light conditions.
- Ensures security (e.g., face presence check).
- Confirms UI responsiveness across various systems.

6.2 Testing Details

We conducted the following testing strategies:

- Manual Testing: For GUI interaction, score display, and gesture recognition flow.
- Functional Testing: For each module (login, gesture recognition, question handling).
- Scenario Testing: Under low light, poor camera quality, and presence of multiple users.
- Performance Testing: On high-end and low-end systems to analyze responsiveness.

Module	Type of Testing	Observed Behavior	Module
Login Module	Input validation, credential check	Properly handled errors and valid inputs	Login Module
Gesture Selection	Real-time gesture-to-option mapping	Mostly accurate; minor errors in fast motion	Gesture Selection
Face Detection	Single/multiple face scenarios	Single face: reliable, Multi-face: inconsistent	Face Detection
Lighting Conditions	Low and high light	In low light, gesture tracking dropped	Lighting Conditions
GUI Responsiveness	Tkinter performance under load	Lag on older machines	GUI Responsiveness
Module	Type of Testing	Observed Behavior	Module
Login Module	Input validation, credential check	Properly handled errors and valid inputs	Login Module

Table 6.1:Test Details

6.3 Test Cases

Test cases play a critical role in the software testing process by providing a systematic approach to verify that the software application behaves as expected under various conditions. A test case is a documented set of actions, inputs, execution conditions, and expected results designed to validate a particular feature or function of the software. In the context of this project, where gesture-based input and real-time interaction are key components, test cases ensure that each part of the system—from hand tracking and face detection to answer selection and score calculation—functions accurately and reliably. Each test case includes essential details such as a unique ID, a clear description, defined input values, steps for execution, and the anticipated outcome. This helps in identifying whether the actual system behavior matches the intended functionality. Well-designed test cases not only cover typical usage scenarios (positive testing) but also include edge cases and invalid conditions (negative testing), thereby ensuring the robustness of the

application. In gesture-controlled environments like this one, test cases are particularly important for verifying the correctness of hand gesture recognition under various lighting conditions, user movements, and hardware limitations. Overall, test cases provide confidence in the quality of the system and help in maintaining consistency and reliability throughout the development lifecycle.

Test Case ID	**Test Case Description**	**Input Action**	**Expected Output**	**Status**
TC01	Valid login	Enter correct username and password	Login successful	Pass
TC02	Invalid login	Leave one or more fields blank	Show error message	Pass
TC03	Correct gesture selection (Option A)	Hover red hand cursor over Option A	Option A highlighted and selected	Pass
TC04	Incorrect gesture selection due to fast hand movement	Move hand too quickly across options	Wrong option may be selected	Fail
TC05	Face not detected	User leaves webcam frame	Warning: "Face Not Detected"	Pass
TC06	Multiple faces in webcam frame	Another person enters view	No warning or block triggered	Fail
TC07	Gesture recognition in low-light	Use gestures in dark room	Red cursor fails to respond or disappears	Fail
TC08	Score display after quiz completion	Complete quiz using gesture selections	Accurate score displayed at end	Pass
TC09	System responsiveness on low-end laptop	Run program on older laptop (2 GB RAM, i3 processor)	Lag in gesture tracking and GUI	Fail

Table.6.2.Test Cases

CHAPTER 7

OUTPUT SCREENSHOTS

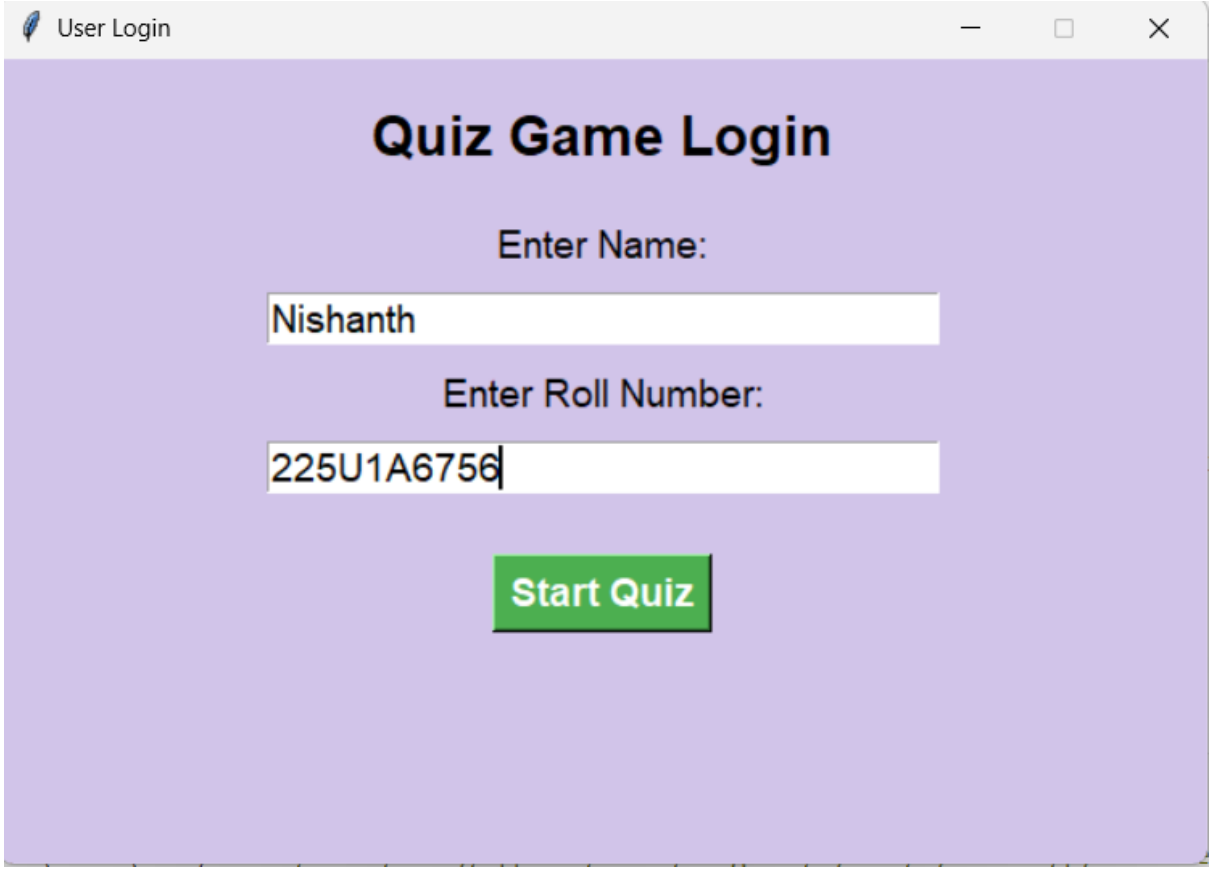

Fig.7.1.login

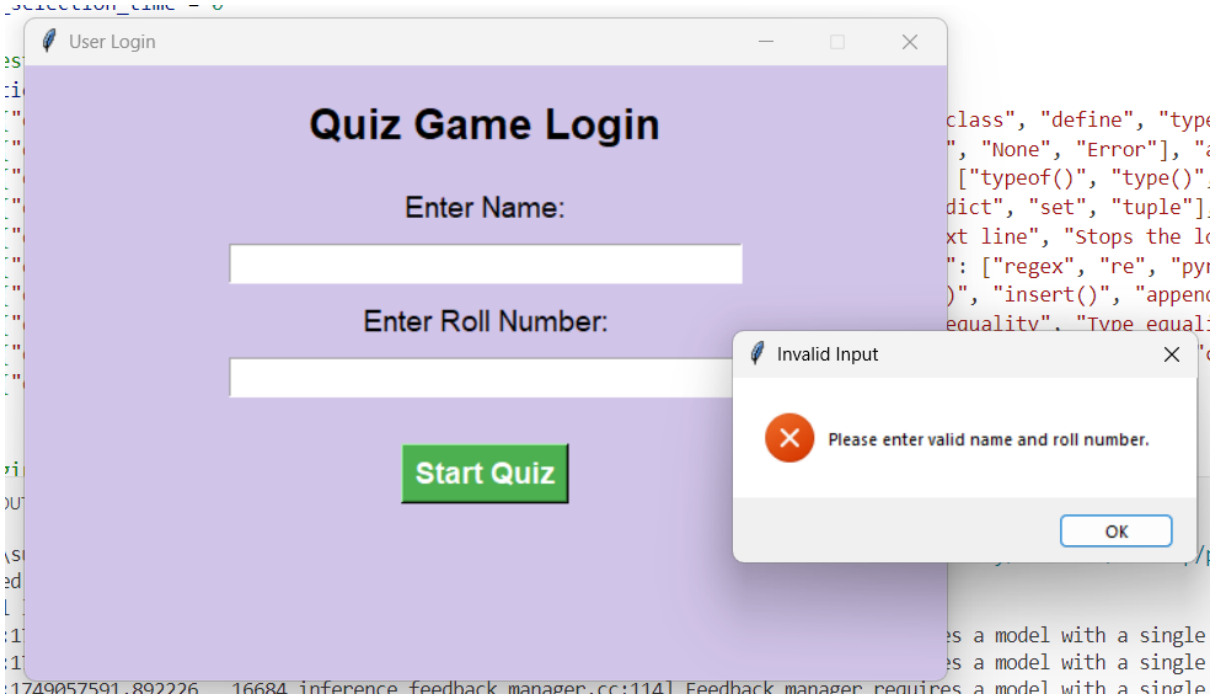

Fig.7.2.Invalid login

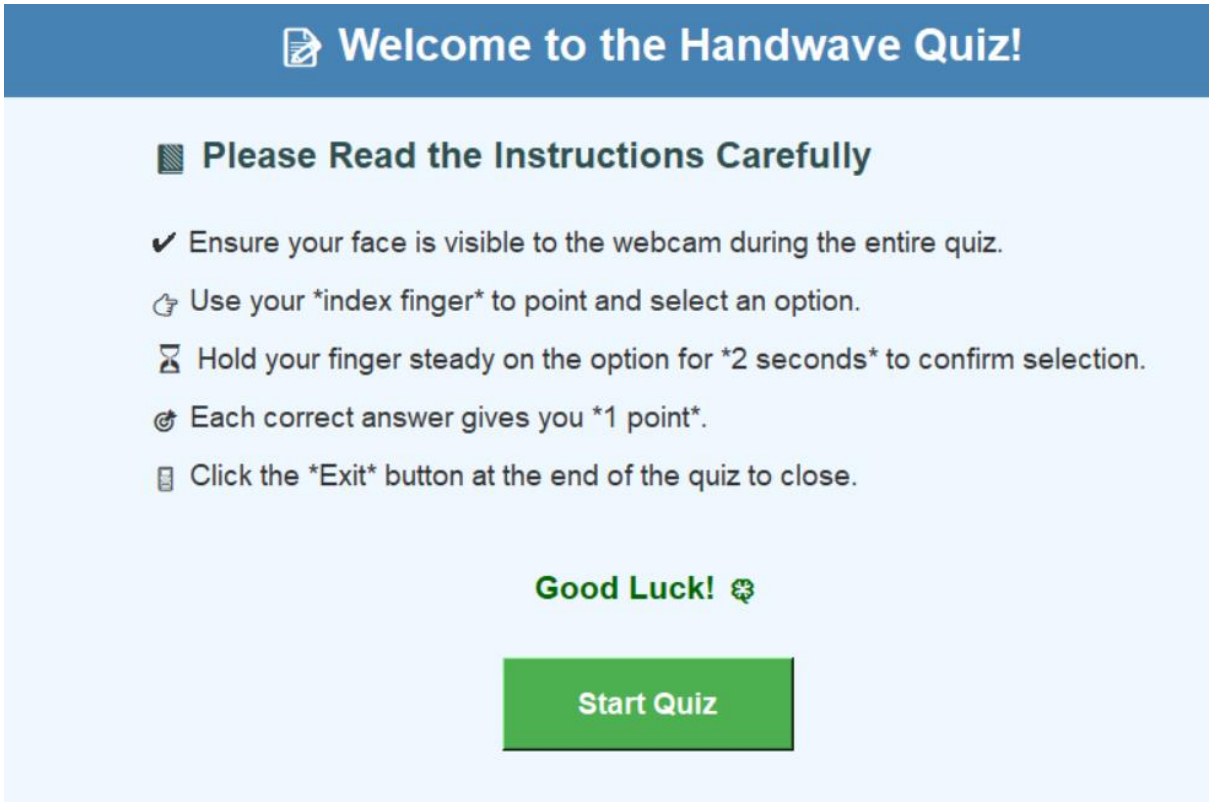

Fig.7.3.Instructions

Fig.7.4.Face not detected

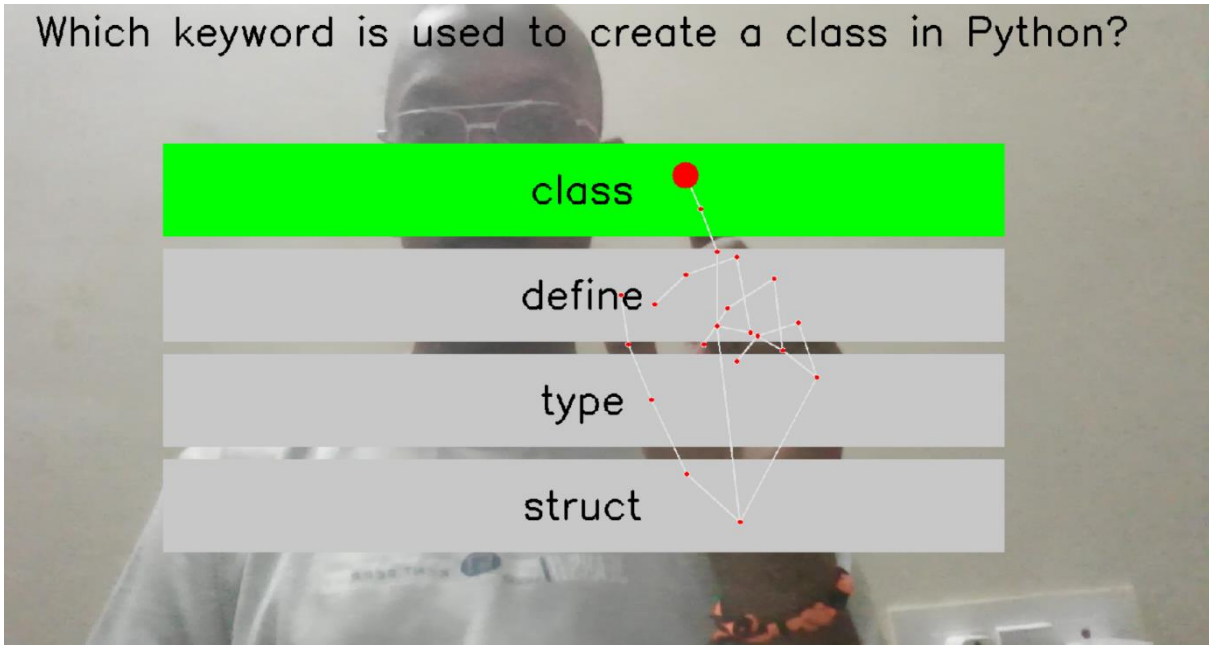

Fig.7.5.Selecting Answers

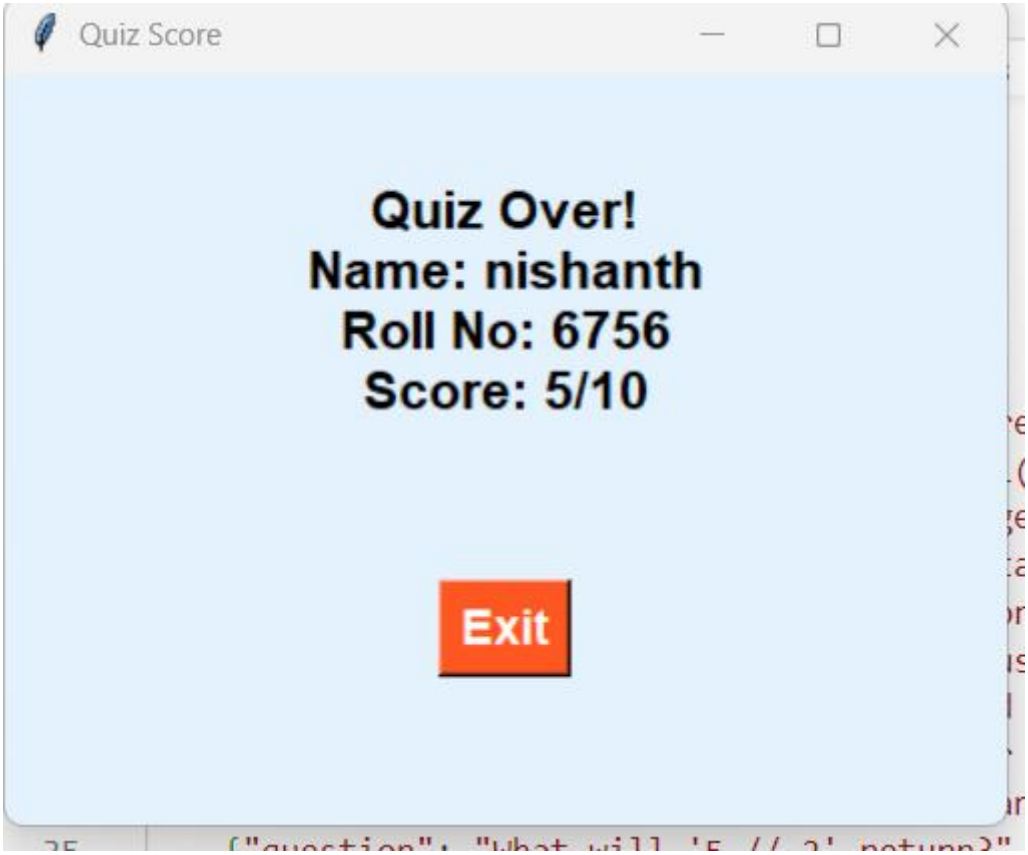

Fig.7.6.Exit and Final Score

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The HandWave Quiz project successfully introduces a novel, contactless quiz system that leverages hand gesture recognition to enhance user interaction and accessibility. By integrating MediaPipe's hand tracking and face detection capabilities with OpenCV's image processing techniques, the system efficiently interprets users' hand gestures to select quiz answers, eliminating the need for traditional input devices like mouse or keyboard.

The development of a user-friendly GUI using Tkinter further enriches the experience by providing clear visual feedback and easy navigation. The system demonstrated effective real-time response and accuracy in recognizing predefined gestures, confirming the feasibility of using computer vision techniques in interactive educational applications.

This approach addresses contemporary needs for hygiene and contactless interfaces, especially relevant in scenarios such as remote learning and pandemic-affected environments. Furthermore, the project lays a foundation for future enhancements in gesture-based human-computer interaction, showing potential to transform conventional quiz formats into more engaging and inclusive experiences.

8.2 Future Work

Despite the successful implementation, there are several promising directions to improve and expand the HandWave Quiz system:

- Advanced Gesture Recognition Models: Current gesture detection relies on predefined hand landmarks; integrating deep learning models could allow recognition of more complex and subtle gestures, improving accuracy and user
- flexibility.
- Adaptive Learning and Personalization: Incorporate machine learning algorithms to adapt quiz difficulty based on user performance, creating a personalized learning environment.

- Cross-Platform Support: Port the application to mobile platforms (Android/iOS) and web browsers to increase accessibility and reach a wider audience.
- Voice and Gesture Multimodal Interaction: Combine gesture recognition with voice commands to create a multimodal interface that caters to various user preferences and accessibility needs.
- Enhanced Security and Authentication: Integrate biometric authentication, such as facial recognition or fingerprint scanning, to secure user accounts and prevent unauthorized access.
- Expanded Content and Question Formats: Add support for varied question types, such as multiple selections, drag-and-drop, audio/video-based questions, and timed quizzes to increase engagement and variety.
- Real-Time Feedback and Analytics: Implement detailed analytics dashboards for users and educators to track performance trends, common mistakes, and gesture accuracy, enabling data-driven improvements.
- Multi-User and Collaborative Features: Enable simultaneous multi-user interactions for group quizzes, competitions, and collaborative learning sessions.
- Accessibility Improvements: Integrate accessibility options such as screen readers, customizable UI themes, and alternative input methods to support users with disabilities.

CHAPTER 9

REFERENCES

1. Hax, D. R. T., Penava, P., Krodel, S., Razova, L., & Buettner, R. (2024). **A Novel Hybrid Deep Learning Architecture for Dynamic Hand Gesture Recognition**. *IEEE Access*, Vol. 12, pp. 28761–28773. DOI: 10.1109/ACCESS.2024.3365274
2. MediaPipe by Google – https://google.github.io/mediapipe/
3. OpenCV Library – https://opencv.org/
4. Tkinter – Python Standard GUI Package – https://docs.python.org/3/library/tk.html
5. Python Software Foundation – https://www.python.org/
6. Towards Data Science – Blogs on Gesture Recognition – https://towardsdatascience.com
7. Sharma, S., & Singh, S. (2021). "Vision-based hand gesture recognition using deep learning for the interpretation of sign language." *Expert Systems with Applications*, 182, 115657.
8. Alzubaidi, L., et al. (2023). “A survey on deep learning tools dealing with data scarcity.” *Journal of Big Data*, 10(1), 46.